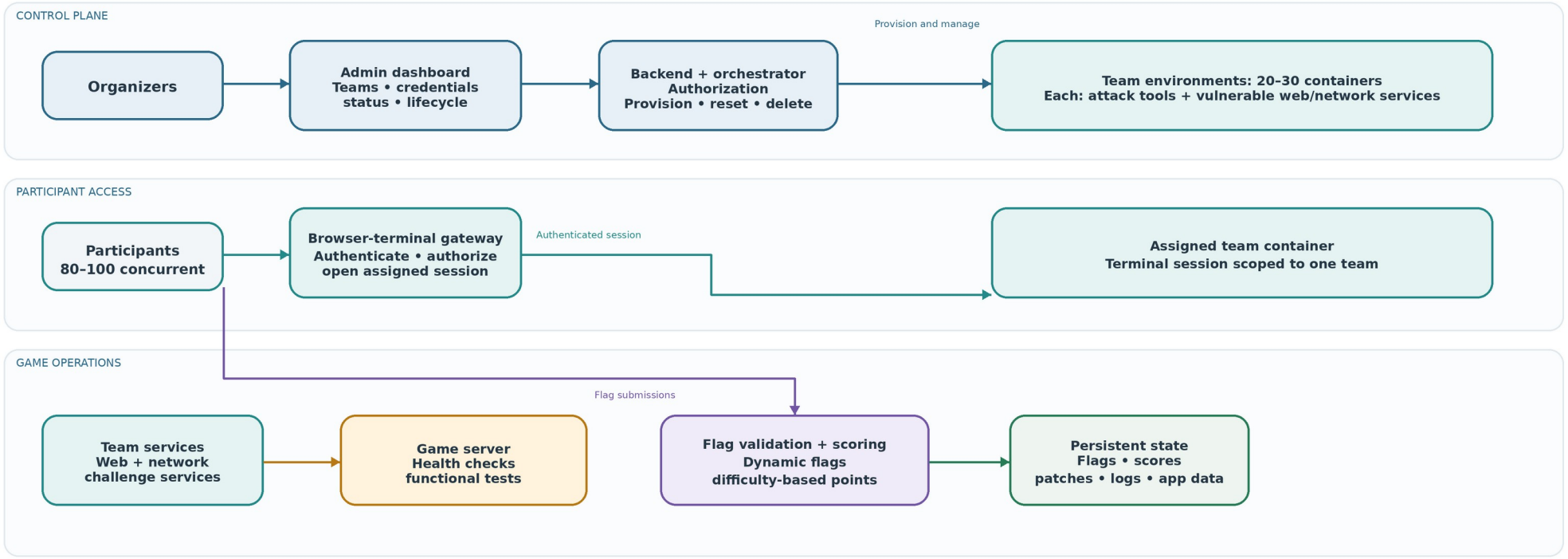
DEFCON 2.0

SYSTEM REQUIREMENTS / DIAGRAM-FIRST EDITION

Version 1.2 • 13 October 2026 • Confirmed decisions are recorded below; unresolved items are on the final page.

20–30 teams Expected team count	**80–100 users** Simultaneous user target	**1 container/team** Tools + vulnerable services
Browser terminal Organizer-issued credentials	**Cross-team attacks** Only intended competition targets	**Attack-only scoring** Dynamic flags; points by difficulty

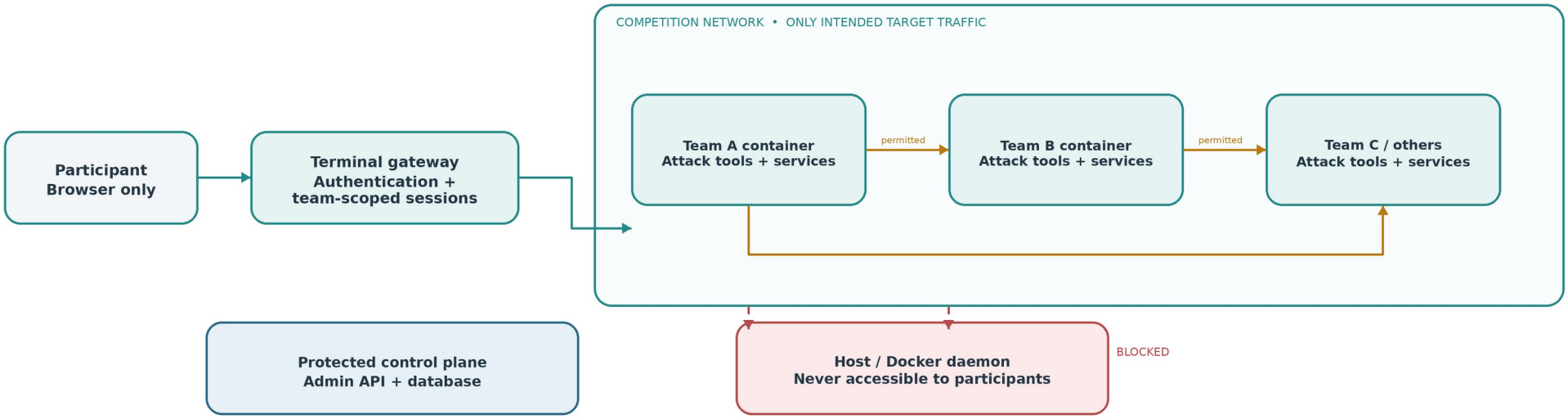
System at a glance



Key implementation constraint

Exactly one container per team at all times, same-container replacement, automatic rollback, and near-zero downtime must be reconciled before deployment implementation is finalized.

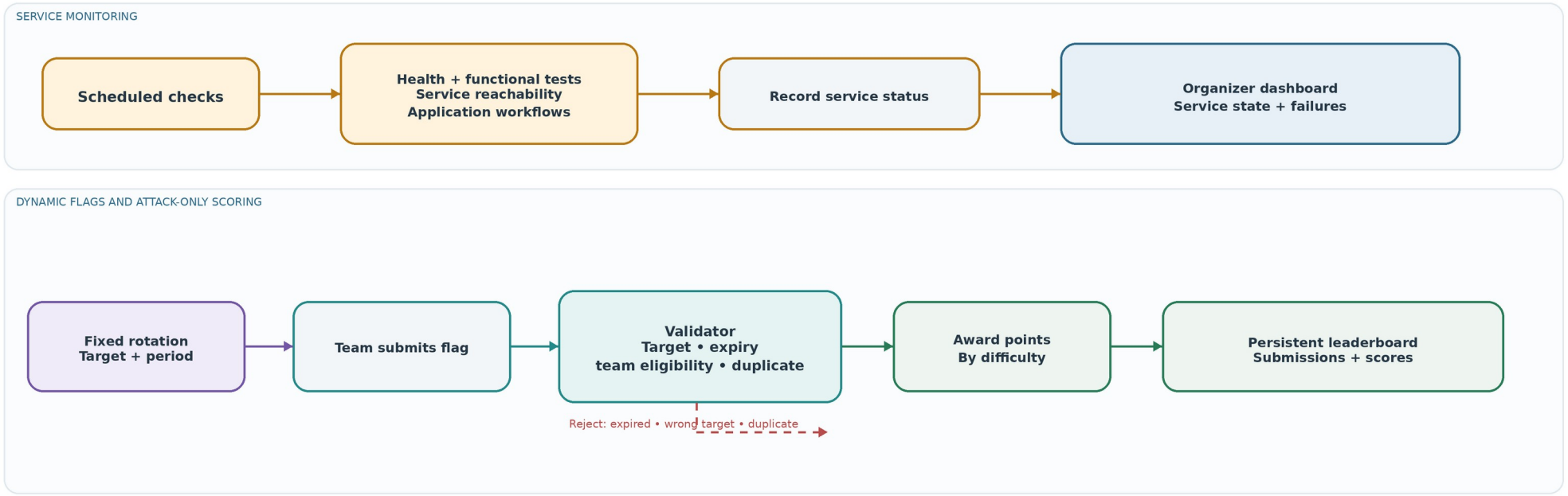
1. Architecture and network boundaries



Component responsibilities

Admin + orchestrator Create/update/delete teams, manage credentials, provision environments and execute lifecycle actions.	**Browser-terminal gateway** Authenticate sessions and restrict each participant to their assigned team.
Team containers Contain attack tools and vulnerable web/network services; Nmap and Netdiscover are initial tools.	**Competition networking** Allow attacks to intended other-team targets; block host, Docker daemon, control plane and unrelated external access.
Restricted privileges No host privileges or Docker socket access. Apply per-team CPU, memory, process and storage limits.	**Existing server** Hosting choice is confirmed, but capacity is unverified until inspected and load-tested.

2. Game server, flags and scoring

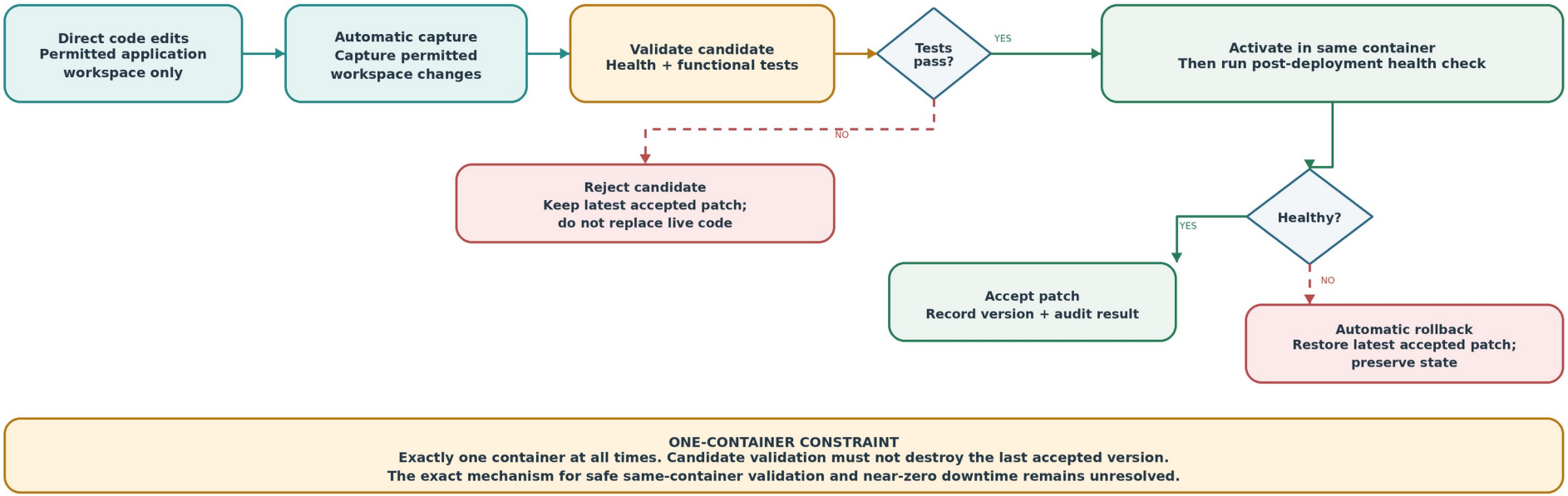


Health checks Detect stopped or unreachable services.	**Functional tests** Verify application workflows, not just open ports.	**Fixed rotation** Flags rotate on a fixed schedule.
Target-specific flags Unique per target and rotation period.	**Validation** Reject expired flags and flags submitted for the wrong target.	**Duplicate protection** A flag scores at most once per attacking team.

Scoring

Attack-only: no points for patching or service availability. Point values vary by challenge difficulty; exact values remain unresolved.

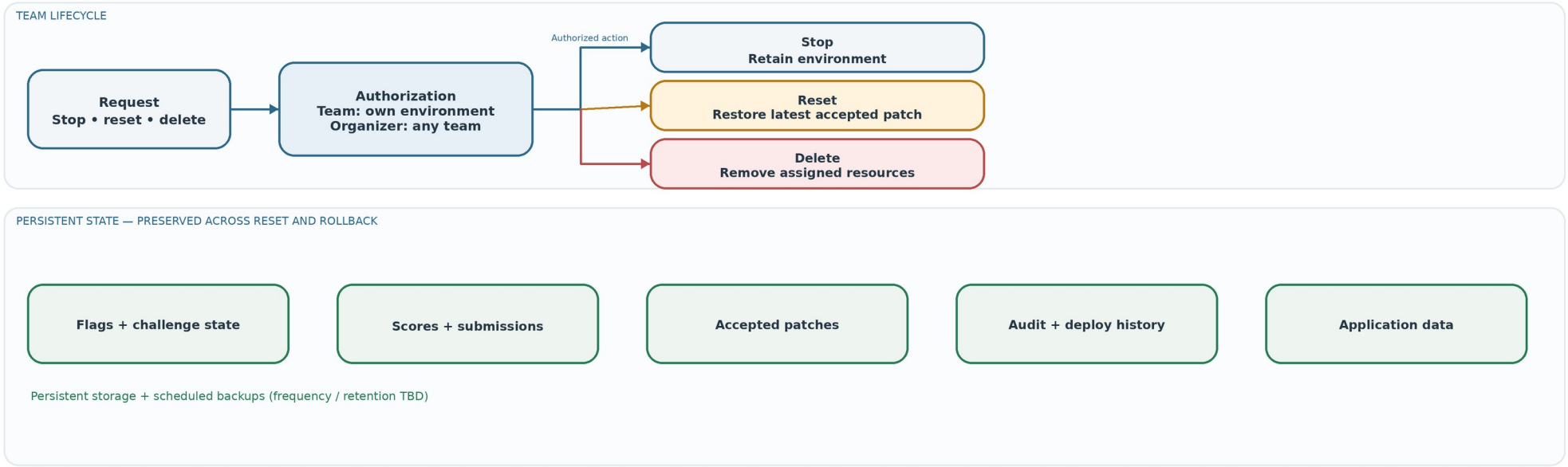
3. Patch capture, validation and rollback



Confirmed deployment behavior

Direct changes Teams edit permitted application files inside their container.	**Automatic capture** Capture all permitted application-workspace changes into a candidate.	**Same-container rule** Exactly one container must exist at all times; same-container replacement is selected.
Failure handling Automatically roll back failed deployments and verify recovery.	**Reset baseline** Restore the latest accepted patch, not the original vulnerable baseline.	**Near-zero downtime** Target only; it is not yet guaranteed by the selected deployment constraint.

4. State, lifecycle and operations



Persistent data

Flags + challenge state Rotation and challenge progress.	**Scores + submissions** Authoritative records of submissions and points.	**Accepted patches** Recoverable accepted application versions.
Audit + deployment history Actions, timestamps, validation and rollback outcomes.	**Application data** Data required by service workflows.	**Backups** Persistent storage + scheduled backups; schedule and retention unresolved.

Operational rules

• Teams reset their own environment; organizers may reset any team, with separate permissions. • Reset, stop and delete are distinct. • Reset/rollback must preserve all five data categories. • Idle terminal sessions expire automatically (timeout TBD). • Health checks run automatically at a less-frequent interval (interval TBD). • Container root privileges are restricted; host and Docker-daemon access are prohibited.

5. Unresolved questions

Only unresolved values or implementation choices are listed here; no values are assumed.

OPEN ITEM	QUESTION TO RESOLVE
Deployment mechanism	How can a candidate be validated and the last accepted version retained with exactly one container at all times, while targeting near-zero downtime?
Existing server capacity	What CPU, RAM, storage, network capacity and current workload are available?
Flag timing	What is the rotation interval, and how are submissions handled at the rotation boundary?
Difficulty points	What point values apply to easy, medium and hard challenges?
Operations	What exact idle-session timeout and health-check interval should be configured?
Backup policy	What are backup frequency, retention period and restore-test schedule?
Privilege escalation	How will privilege-escalation challenges work under the restricted-privilege policy?
Challenge catalogue	Which exact challenges, targets, flags and functional tests belong to each difficulty tier?

6. Suggestions: reduce repetition and simplify development

Modular monolith first One backend API and shared database; separate modules without starting with microservices.	**One team lifecycle state machine** Reuse states across admin UI, orchestrator, reset and monitoring.
One standard team image Maintain one image; inject team-specific settings at provisioning.	**One challenge manifest** Store target, difficulty, points, flag rules and tests in one format.
Single authoritative scorer Centralize flag validation; use transactional duplicate protection.	**Separate state from runtime** Keep scores, flags, accepted patches, logs and durable app data outside disposable container state.
Shared health-check framework One scheduler, shared test interface and one alert path.	**Idempotent automation** Provision, reset, deploy and rollback should be safe to retry.
Prototype then scale Complete one team environment end-to-end before scaling to 20–30 teams.	**Measure before sizing** Inspect current hardware, set resource limits, and load-test with 100 concurrent users.